Übungsblatt 10

Dependency Injection, Testing und Persistenz

5430UE Web and Data Engineering

24. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundlegendes Verständnis der Konzepte Inversion of Control (IoC) und Dependency Injection (DI) im Spring Framework.
- Einblick in verschiedene Testarten sowie die Test-Pyramide.
- Grundlegendes Verständnis der Dateninitialisierung beim Anwendungsstart und der Vererbungsabbildung mit JPA.

Dependency Injection (DI) und Inversion of Control (IoC)

Dependency Injection (DI) und Inversion of Control (IoC) (1/3)

Eines der Kernkonzepte des Spring Frameworks ist die *Dependency Injection* (DI) als konkrete Umsetzung des *Inversion of Control* (IoC) Prinzips.

1. Erklären Sie in eigenen Worten, was man unter Dependency Injection versteht und wie das Konzept grundlegend funktioniert. Gehen Sie dabei kurz auf die Begriffe *Inversion of Control (IoC)*, *IoC-Container* und *Spring Bean* ein.



Dependency Injection (DI) und Inversion of Control (IoC) (2/3)

Grundlage für die folgenden beiden Teilaufgaben stellen die nachfolgend gegebenen Implementierungen eines `OrderService` dar.

```
// Version A
@Service
public class OrderService {
  private final EmailService email = new EmailService();
  public void placeOrder(Order o) { email.send(o); }
}

// Version B
@Service
public class OrderService {
  private final EmailService email;

  public OrderService(EmailService email) {
    this.email = email;
  }

  public void placeOrder(Order o) { email.send(o); }
}
```



Dependency Injection (DI) und Inversion of Control (IoC) (3/3)

2. Welche der beiden Versionen ist besser testbar? Begründen Sie Ihre Antwort.

3. Was macht Spring im Hintergrund automatisch bei Version B, wenn die Anwendung gestartet wird?

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (1/3)

1. **Konzept der Dependency Injection:**

-
-
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (1/3)

1. **Konzept der Dependency Injection:**

- **Dependency Injection (DI):** Ein Entwurfsmuster, bei dem ein Objekt seine Abhängigkeiten (Dependencies) nicht selbst erstellt (z. B. per `new`), sondern diese von außen „injiziert“ (übergeben) bekommt. Dies fördert lose Kopplung.
-
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (1/3)

1. **Konzept der Dependency Injection:**

- **Dependency Injection (DI):** Ein Entwurfsmuster, bei dem ein Objekt seine Abhängigkeiten (Dependencies) nicht selbst erstellt (z. B. per `new`), sondern diese von außen „injiziert“ (übergeben) bekommt. Dies fördert lose Kopplung.
- **Inversion of Control (IoC):** Bezeichnet die Umkehrung des Kontrollflusses. Bei der klassischen Programmierung steuert der eigene Code den Ablauf und erzeugt benötigte Objekte selbst. Bei IoC gibt man diese Kontrolle an ein übergeordnetes Framework ab (das sogenannte *Hollywood-Prinzip*: „Don't call us, we'll call you“). *Beispiel:* Statt dass ein Auto-Objekt sich selbst mühsam einen Motor (per `new Motor()`) zusammenbaut, wartet es darauf, dass das Framework (der Mechaniker) ihm einen fertigen Motor von außen einsetzt.
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (1/3)

1. **Konzept der Dependency Injection:**

- **Dependency Injection (DI):** Ein Entwurfsmuster, bei dem ein Objekt seine Abhängigkeiten (Dependencies) nicht selbst erstellt (z. B. per `new`), sondern diese von außen „injiziert“ (übergeben) bekommt. Dies fördert lose Kopplung.
- **Inversion of Control (IoC):** Bezeichnet die Umkehrung des Kontrollflusses. Bei der klassischen Programmierung steuert der eigene Code den Ablauf und erzeugt benötigte Objekte selbst. Bei IoC gibt man diese Kontrolle an ein übergeordnetes Framework ab (das sogenannte *Hollywood-Prinzip*: „Don't call us, we'll call you“). *Beispiel:* Statt dass ein Auto-Objekt sich selbst mühsam einen Motor (per `new Motor()`) zusammenbaut, wartet es darauf, dass das Framework (der Mechaniker) ihm einen fertigen Motor von außen einsetzt.
- **IoC-Container & Beans:** In Spring übernimmt der IoC-Container (`ApplicationContext`) diese Rolle. Er durchsucht die Anwendung beim Start, erstellt die benötigten Objekte (die in Spring als *Beans* bezeichnet werden) und injiziert sie automatisch dort, wo sie gebraucht werden.

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (2/3)

2. **Testbarkeit der Versionen: Version B ist deutlich testbarer.**

-
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (2/3)

2. **Testbarkeit der Versionen: Version B ist deutlich testbarer.**

- In **Version A** ist der `EmailService` fest im Code verankert (starke Kopplung). Wenn man nun den `OrderService` isoliert auf Fehler prüfen möchte, würde bei jedem Versuch unweigerlich eine echte E-Mail gesendet bzw. ein echter `EmailService` genutzt werden.
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (2/3)

2. **Testbarkeit der Versionen: Version B ist deutlich testbarer.**

- In **Version A** ist der `EmailService` fest im Code verankert (starke Kopplung). Wenn man nun den `OrderService` isoliert auf Fehler prüfen möchte, würde bei jedem Versuch unweigerlich eine echte E-Mail gesendet bzw. ein echter `EmailService` genutzt werden.
- In **Version B** wird die Abhängigkeit flexibel über den Konstruktor von außen übergeben. Dadurch kann man beim Testen ganz einfach ein Dummy-Objekt (Mock-Objekt) übergeben, um die Logik zu prüfen, ohne dass echte Fremdsysteme angesprochen werden.

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (3/3)

3. **Automatisches Verhalten von Spring (Version B):**

-
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (3/3)

3. **Automatisches Verhalten von Spring (Version B):**

- Spring erkennt beim Starten die Annotation `@Service` und weiß, dass es eine Bean vom Typ `OrderService` verwalten muss.
-

Dependency Injection (DI) und Inversion of Control (IoC) — Lösung (3/3)

3. **Automatisches Verhalten von Spring (Version B):**

- Spring erkennt beim Starten die Annotation `@Service` und weiß, dass es eine Bean vom Typ `OrderService` verwalten muss.
- Da der Konstruktor einen `EmailService` verlangt, sucht der IoC-Container nach einer passenden Bean für diesen Service, erstellt diese (falls noch nicht geschehen) und reicht sie automatisch als Parameter in den Konstruktor hinein (*Constructor Injection*).

Die Test-Pyramide

Die Test-Pyramide

Beschreiben Sie die drei Ebenen der Test-Pyramide für eine (Spring Boot) Anwendung.

1. Was testet jede Ebene? Nennen Sie ein konkretes Beispiel.
2. Wie verhält sich die empfohlene Anzahl der Tests sowie deren Ausführungsgeschwindigkeit von der untersten zur obersten Ebene?



Die Test-Pyramide — Lösung (1/3)

1. **Ebenen der Test-Pyramide:**

-
-
-

Die Test-Pyramide — Lösung (1/3)

1. **Ebenen der Test-Pyramide:**

- **Unit Tests (Basis):** Testen isolierte Komponenten oder einzelne Klassen unabhängig vom Rest des Systems (Abhängigkeiten werden mit Mocks simuliert).
  Beispiel: Ein Test, der prüft, ob eine Methode zur Preisberechnung den korrekten Rabatt anwendet (z. B. mit JUnit).
-
-

Die Test-Pyramide — Lösung (1/3)

1. **Ebenen der Test-Pyramide:**

- **Unit Tests (Basis):** Testen isolierte Komponenten oder einzelne Klassen unabhängig vom Rest des Systems (Abhängigkeiten werden mit Mocks simuliert).
  Beispiel: Ein Test, der prüft, ob eine Methode zur Preisberechnung den korrekten Rabatt anwendet (z. B. mit JUnit).

- **Integration Tests (Mitte):** Testen das Zusammenspiel mehrerer Systemkomponenten oder die Anbindung an externe Ressourcen (z. B. Datenbanken).
  Beispiel: Ein Test für den `StudentController`. Es wird geprüft, ob bei einem `POST /students`-Request die korrekten JSON-Daten an den Service weitergereicht und der HTTP-Status `201 Created` korrekt zurückgegeben wird. Der Service selbst wird hierbei oft durch einen Mock ersetzt, um sich rein auf die Controller-Logik zu konzentrieren.

-

Die Test-Pyramide — Lösung (1/3)

1. **Ebenen der Test-Pyramide:**

- **Unit Tests (Basis):** Testen isolierte Komponenten oder einzelne Klassen unabhängig vom Rest des Systems (Abhängigkeiten werden mit Mocks simuliert).
  Beispiel: Ein Test, der prüft, ob eine Methode zur Preisberechnung den korrekten Rabatt anwendet (z. B. mit JUnit).

- **Integration Tests (Mitte):** Testen das Zusammenspiel mehrerer Systemkomponenten oder die Anbindung an externe Ressourcen (z. B. Datenbanken).
  Beispiel: Ein Test für den `StudentController`. Es wird geprüft, ob bei einem `POST /students`-Request die korrekten JSON-Daten an den Service weitergereicht und der HTTP-Status `201 Created` korrekt zurückgegeben wird. Der Service selbst wird hierbei oft durch einen Mock ersetzt, um sich rein auf die Controller-Logik zu konzentrieren.

- **End-to-End (Spitze):** Testen das komplette System von der Benutzeroberfläche bis zur Datenbank unter realen Bedingungen (Black-Box-Test).
  Beispiel: Ein Test, der die Anwendung vollständig startet, einen echten Datensatz per HTTP-Request an den Endpunkt `/students` sendet, diesen in der Datenbank speichert und anschließend per `GET` den korrekten Status abruft.

Die Test-Pyramide — Lösung (2/3)

2. **Verhältnis (Anzahl & Geschwindigkeit):**

-
-

Die Test-Pyramide — Lösung (2/3)

2. **Verhältnis (Anzahl & Geschwindigkeit):**

- **Anzahl der Tests:** Von unten nach oben nimmt die empfohlene Anzahl der Tests ab. Die Basis sollte aus sehr vielen Unit Tests bestehen, gefolgt von einer moderaten Anzahl an Integrationstests und nur wenigen End-to-End-Tests.
-

Die Test-Pyramide — Lösung (2/3)

2. **Verhältnis (Anzahl & Geschwindigkeit):**

- **Anzahl der Tests:** Von unten nach oben nimmt die empfohlene Anzahl der Tests ab. Die Basis sollte aus sehr vielen Unit Tests bestehen, gefolgt von einer moderaten Anzahl an Integrationstests und nur wenigen End-to-End-Tests.
- **Ausführungsgeschwindigkeit & Komplexität:** Von unten nach oben nimmt die Ausführungszeit sowie die Komplexität zu. Unit Tests laufen in Millisekunden, End-to-End-Tests dauern oft Minuten und sind wartungsintensiver sowie fehleranfälliger.

Datenbank-Initialisierung mit dem CommandLineRunner



Datenbank-Initialisierung mit dem CommandLineRunner

Bei der Entwicklung einer Spring Boot-Anwendung ist es oft hilfreich, wenn beim Start automatisch Testdaten in die (In-Memory-)Datenbank geladen werden, damit man direkt mit der Entwicklung der Endpunkte oder der UI beginnen kann.

Laden Sie sich dazu das Projekt `CommandLineRunnerDemo.zip` aus Stud.IP herunter. Das Projekt enthält bereits eine fertige Entität `Product` (mit Name und Preis) sowie das zugehörige `ProductRepository`.

1. Was ist ein `CommandLineRunner` in Spring Boot und zu welchem exakten Zeitpunkt im Lebenszyklus der Anwendung wird er ausgeführt?
2. Im Projekt finden Sie die leere Klasse `DataInitializer`. Erweitern Sie diese Klasse so, dass sie beim Starten der Anwendung automatisch drei verschiedene Produkte in der Datenbank speichert. (Anforderungen an den Code siehe Folgeseite)

Anforderungen an die Code-Implementierung

- Nutzen Sie Dependency Injection, um das `ProductRepository` einzubinden.
- Geben Sie unmittelbar vor und nach dem Speichern der Daten jeweils eine Meldung über `System.out.println(...)` auf der Konsole aus (z. B. „*Starte Initialisierung...*“), damit Sie den Zeitpunkt der Ausführung in den Start-Logs von Spring gut erkennen können.



Datenbank-Initialisierung mit dem CommandLineRunner — Lösung (1/3)

1. **Was ist der CommandLineRunner?**

- Der `CommandLineRunner` ist ein Interface in Spring Boot, das genau eine Methode vorgibt: `run(String... args)`.
- **Zeitpunkt der Ausführung:** Wenn eine Klasse dieses Interface implementiert (und als Spring Bean registriert ist), führt Spring die `run`-Methode automatisch aus, *nachdem* der komplette `ApplicationContext` (inklusive Datenbankverbindung, Controller, etc.) vollständig aufgebaut und gestartet wurde, aber noch bevor die Applikation beginnt, reguläre Nutzeranfragen abzuarbeiten.

Datenbank-Initialisierung mit dem CommandLineRunner — Lösung (2/3)

2. Implementierung des DataInitializers: - Teil 1

```
package wde.testdemo;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component // Sagt Spring, dass diese Klasse als Bean verwaltet werden soll
public class DataInitializer implements CommandLineRunner {
  private final ProductRepository productRepository;

  public DataInitializer(ProductRepository productRepository) {
    this.productRepository = productRepository;
  }
```

Datenbank-Initialisierung mit dem CommandLineRunner — Lösung (3/3)

2. Implementierung des DataInitializers: - Teil 2

```
@Override
public void run(String... args) throws Exception {
  System.out.println("========================================");
  System.out.println("Starte Daten-Initialisierung...");

  Product p1 = new Product("Laptop", 999.99);
  productRepository.save(p1);
  System.out.println("Eingefuegt: " + p1.getName());

  Product p2 = new Product("Mouse", 29.99);
  productRepository.save(p2);
  System.out.println("Eingefuegt: " + p2.getName());

  Product p3 = new Product("Keyboard", 79.99);
  productRepository.save(p3);
  System.out.println("Eingefuegt: " + p3.getName());

  System.out.println("Alle Produkte erfolgreich in die DB geladen!");
```

JPA Vererbungsstrategien

JPA Vererbungsstrategien

Sie modellieren eine Klassenhierarchie für das Personalwesen einer Hochschule: Eine abstrakte Basisklasse `Person` (Attribute: `id`, `name`), von der die Klassen `Student` (zusätzlich: `semester: int`) und `Lehrkraft` (zusätzlich: `lehrstuhl: String`) erben.

1. Skizzieren Sie die drei JPA-Vererbungsstrategien (`SINGLE_TABLE`, `JOINED`, `TABLE_PER_CLASS`). Erklären Sie jeweils in einem Satz, wie die Tabellenstruktur aussieht, nennen Sie Vor- und Nachteile und geben Sie an, wie die Annotation an der Basisklasse `Person` aussehen muss.

2. Welche Strategie würden Sie für dieses konkrete Szenario wählen und warum?

JPA Vererbungsstrategien — Lösung (1/9)

1. SINGLE_TABLE (Default-Strategie)

- **Erklärung:** Alle Klassen der Hierarchie werden in einer einzigen, gemeinsamen Datenbanktabelle gespeichert. Eine automatische Diskriminator-Spalte (`dtype`) kennzeichnet den Typ.
- **Vorteile:** Extrem performant, da keine `JOIN`s für Abfragen über die gesamte Hierarchie nötig sind.
- **Nachteile:** Spalten von Unterklassen müssen zwingend `NULL` erlauben (`NOT NULL`-Constraints sind unmöglich), was die Datenintegrität schwächt.
- **Datenbank-Struktur:**

```
Tabelle: PERSON
+----+----------+----------+------------+-----------+
| id | name     | semester | lehrstuhl  | dtype     |
+----+----------+----------+------------+-----------+
| 1  | Anna     | 4        | NULL       | student   |
| 2  | Prof. X  | NULL     | Informatik | lehrkraft |
+----+----------+----------+------------+-----------+
```

JPA Vererbungsstrategien — Lösung (2/9)

1. SINGLE_TABLE (Code-Umsetzung)

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "dtype")
public abstract class Person {
  @Id @GeneratedValue private Long id;
  private String name;
}

@Entity
@DiscriminatorValue("student")
public class Student extends Person {
  private int semester;
}

@Entity
@DiscriminatorValue("lehrer")
public class Lehrkraft extends Person {
  private String lehrstuhl;
}
```



JPA Vererbungsstrategien — Lösung (3/9)

2. JOINED

- **Erklärung:** Jede Klasse erhält eine eigene Tabelle. Unterklassen-Tabellen speichern nur ihre spezifischen Attribute und verweisen über die ID (Fremdschlüssel) auf die Basistabelle.
- **Vorteile:** Sauberes, voll-normalisiertes Schema. `NOT NULL`-Constraints sind möglich.
- **Nachteile:** Schlechtere Lese-Performance, da `JOIN`s mit der Basistabelle nötig sind.

JPA Vererbungsstrategien — Lösung (4/9)

2. JOINED

- **Datenbank-Skizze:**

```
Tabelle: PERSON
+----+----------+
| id | name     |  <-- Das ist die "referencedColumnName" (id)
+----+----------+
| 1  | Anna     |
| 2  | Prof. X  |
+----+----------+

Tabelle: STUDENT
+----+----------+
| id | semester |  <-- Spalten id (als Foreign Key auf PERSON.id)
+----+----------+
| 1  | 4        |
+----+----------+

Tabelle: LEHRKRAFT
+-----+------------+
| id  | lehrstuhl  |  <-- Spalte id (als Foreign Key auf PERSON.id)
```

JPA Vererbungsstrategien — Lösung (5/9)

2. JOINED

- **Code-Umsetzung (an der Basisklasse):**

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Person {
@Id @GeneratedValue private Long id;
private String name;
}

@Entity
@PrimaryKeyJoinColumn(name = "id", referencedColumnName = "id")
public class Student extends Person {
private int semester;
}

@Entity
@PrimaryKeyJoinColumn(name = "id", referencedColumnName = "id")
public class Lehrkraft extends Person {
private String lehrstuhl;
}
```



JPA Vererbungsstrategien — Lösung (6/9)

3. TABLE_PER_CLASS

- **Erklärung:** Für jede konkrete Unterklasse wird eine eigenständige Tabelle erzeugt, die alle Attribute (auch die geerbten) enthält. Es gibt keine gemeinsame Basistabelle.
- **Vorteile:** Keine `JOIN`s beim Abfragen einer konkreten Unterklasse.
- **Nachteile:** Redundanz; polymorphe Abfragen (z. B. "Lade alle Personen") erfordern extrem ineffiziente `UNION`-Abfragen über alle Tabellen.

JPA Vererbungsstrategien — Lösung (7/9)

3. TABLE_PER_CLASS

Datenbank-Skizze:

```
Tabelle: STUDENT                    Tabelle: LEHRKRAFT
+----+----------+----------+      +----+----------+------------+
| id | name     | semester |      | id | name     | lehrstuhl  |
+----+----------+----------+      +----+----------+------------+
| 1  | Anna     | 4        |      | 2  | Prof. X  | Informatik |
+----+----------+----------+      +----+----------+------------+
```

JPA Vererbungsstrategien — Lösung (8/9)

3. TABLE_PER_CLASS

Code-Umsetzung (an der Basisklasse):

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Person {
  @Id @GeneratedValue private Long id;
  private String name;
}

@Entity
public class Student extends Person {
  private int semester;
}

@Entity
public class Lehrkraft extends Person {
  private String lehrstuhl;
}
```

JPA Vererbungsstrategien — Lösung (9/9)

2. **Empfehlung für das Personalwesen-Szenario:**

- **Empfehlung:** `JOINED` ist hier die beste Wahl. Da die Hierarchie flach ist und die Unterklassen spezifische Pflichtfelder (`semester`, `lehrstuhl`) besitzen, gewinnt man durch `JOINED` echte Typsicherheit auf Datenbankebene (`NOT NULL`-Constraints).

Zusatz zur Abwägung:

- `SINGLE_TABLE` wäre vorzuziehen, wenn sehr viele Abfragen über alle Personen-Typen hinweg laufen und die Performance eine kritische Rolle spielt.
- `TABLE_PER_CLASS` ist selten sinnvoll; am ehesten, wenn gemeinsame Abfragen über alle Personen-Typen hinweg nie vorkommen und Unterklassen getrennt abgefragt werden.

Vertiefung: Die Test-Pyramide in der Praxis (Hands-On)

Vertiefung: Die Test-Pyramide in der Praxis (Hands-On) (1/2)

Hinweis: Diese Aufgabe wird in der Übung je nach zur Verfügung stehender Zeit besprochen.

Laden Sie sich das Projekt `testdemo.zip` aus Stud.IP herunter und öffnen Sie es in Ihrer IDE.

Aufbau des Projekts: Das Projekt simuliert ein stark vereinfachtes Immatrikulationssystem.

- `Student` hält die Daten (hier nur den Namen).
- `StudentService` enthält die Geschäftslogik und prüft, ob der Name gültig ist.
- `StudentController` nimmt HTTP-POST-Anfragen entgegen und ruft den Service auf.

Vertiefung: Die Test-Pyramide in der Praxis (Hands-On) (2/2)

Die zugehörigen Testklassen finden Sie im Projektbaum unter `src/test/java/wde/testdemo`. Die Tests sind lauffähig, jedoch fehlen Erklärungen.

1. **Klassifizierung:** Ordnen Sie die drei Testklassen (`StudentTestA`, `StudentTestB`, `StudentTestC`) der korrekten Ebene der Testpyramide zu (Unit Test, Integration Test, E2E Test). Begründen Sie Ihre Zuordnung kurz anhand der im Code verwendeten Annotationen oder fehlenden Spring-Kontexte.
2. **Code-Verständnis:** Analysieren Sie den Quellcode der drei Testklassen. Ergänzen Sie aussagekräftige (Inline-)Kommentare direkt im Quellcode, die erklären, *was* in den markanten Zeilen passiert (insbesondere bei den Annotationen und Mocking-Befehlen).

Vertiefung: Test-Pyramide in der Praxis — Lösung (1/5)

1. Lösung zu StudentTestA (Unit Test)

- **Klassifizierung: Unit Test**
- **Begründung:** Die Klasse nutzt keinerlei Spring-Annotationen. Der `StudentService` wird ganz klassisch per `new` instanziiert. Es wird reines Java getestet, ohne dass ein Framework hochgefahren werden muss.
- **Kommentierter Quellcode:**

```java
@Test // Markiert die Methode als ausfuehrbaren JUnit-Test
void enroll_withValidName_returnsTrue() {
    Student s = new Student("Max Mustermann");
    // Prueft, ob der Service bei einem gueltigen Namen 'true' zurueckgibt
    assertTrue(service.enroll(s));
}

@Test
void enroll_withEmptyName_throwsException() {
    Student s = new Student("");
    // Prueft, ob beim Uebergeben eines leeren Namens genau diese Exception geworfen wird
    assertThrows(IllegalArgumentException.class, () -> service.enroll(s));
}
```

Vertiefung: Test-Pyramide in der Praxis — Lösung (2/5)

1. Klassifizierung StudentTestB (Integration Test)

- **Klassifizierung: Integration Test**
- **Begründung:** Hier wird das Zusammenspiel der Komponenten in der Web-Schicht getestet (z. B. HTTP-Routing, JSON-Konvertierung und Statuscodes).
- **Die Rolle von `@WebMvcTest`:** Diese Annotation sorgt dafür, dass Spring nicht die gesamte Anwendung, sondern *ausschließlich* den Controller lädt. Um Nebeneffekte zu vermeiden, wird der eigentliche `StudentService` mittels `@MockitoBean` durch eine Attrappe (Mock) ersetzt. Die Anfragen werden über `MockMvc` lediglich simuliert, ohne dass ein echter Netzwerk-Port belegt wird.

Vertiefung: Test-Pyramide in der Praxis — Lösung (3/5)

2. Code-Lösung zu StudentTestB (Integration Test)

```
@WebMvcTest(StudentController.class) // Startet NUR den Web-Layer (Controller)
public class StudentTestB {
  @Autowired
  private MockMvc mockMvc; // Simuliert HTTP-Endpunkte ohne echten Server-Start

  @MockitoBean // Erstellt eine Service-Attrappe und injiziert diese
  private StudentService service;

  @Test
  void createStudent_withValidJson_returnsOk() throws Exception {
    // Mock-Verhalten definieren: enroll liefert bei beliebigem Student true
    when(service.enroll(any(Student.class))).thenReturn(true);

    // Einen simulierten POST-Request senden
    mockMvc.perform(post("/students")
    .contentType(MediaType.APPLICATION_JSON) // Request-Format
    .content("{\"name\": \"Anna\"}")) // Request-Body
    .andExpect(status().isOk()); // Erwartete HTTP-Antwort: 200 OK
  }
```

Vertiefung: Test-Pyramide in der Praxis — Lösung (4/5)

1. Klassifizierung StudentTestC (End-to-End Test)

- **Klassifizierung: End-to-End Test (E2E)**
- **Begründung:** Dieser Test prüft das System als Ganzes unter realen Bedingungen.
- **Die Rolle von `@SpringBootTest`:** Die Annotation `@SpringBootTest(webEnvironment = RANDOM_PORT)` weist Spring an, den kompletten Application-Context (alle Beans) aufzubauen und einen echten Tomcat-Webserver auf einem freien, zufälligen Port zu starten. Mit dem `TestRestTemplate` verhält sich der Test wie ein echter externer Client (z. B. ein Frontend), der über das Netzwerk echte HTTP-Anfragen an den Server sendet.

Vertiefung: Test-Pyramide in der Praxis — Lösung (5/5)

2. Code-Lösung zu StudentTestC (E2E Test)

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class StudentTestC {
  @Autowired
  private TestRestTemplate restTemplate; // Echter HTTP-Client fuer Netzwerkanfragen

  @Test
  void entireApplicationFlow_worksCorrectly() {
    Student s = new Student("Tom");

    // Fuehrt einen echten HTTP-POST-Request gegen die laufende Applikation aus
    ResponseEntity<String> response = restTemplate.postForEntity("/students", s, String.class);

    // Ueberprueft, ob der Server mit 200 OK antwortet
    assertEquals(HttpStatus.OK, response.getStatusCode());
    // Ueberprueft den Response-Body des Controllers
    assertEquals("Erfolgreich immatrikuliert", response.getBody());
  }
}
```

Materialien zum Übungsblatt

- Aufgabe *Datenbank-Initialisierung mit dem CommandLineRunner*: Vorlage CommandLineRunner (`CommandLineRunnerDemo.zip`)
- Aufgabe *Die Test-Pyramide in der Praxis (Hands-On)*: Vorlage Test-Pyramide (`testdemo.zip`)

Lösungen:

- Aufgabe *Datenbank-Initialisierung mit dem CommandLineRunner*: Lösung CommandLineRunner (`CommandLineRunnerDemoLSG.zip`)
- Aufgabe *Die Test-Pyramide in der Praxis (Hands-On)*: Kommentierte Testklassen (`testdemo kommentiert.zip`)